

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Debugging & Optimisation Task

Course Code:	CSA0620	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Precision (BL1, BL2, BL3) Assessment: Assessment Tool 3 – Debugging & Optimisation Task		
Weightage:	40% of CIA	Total Marks:	100 (1 Question × 50 Marks)

CO2 Statement: Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)

Student Name:	VIGNESH.Y	Reg. No.:	192524100
Section / Batch:	2025	Date:	29/08/2026

Instructions to Students

- Answer the question. The question carries 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions	Marks
Debugging & Optimisation Task	Apply asymptotic analysis to nested-loop brute-force algorithms	Q1	100
			100

GRAND TOTAL: 100 Marks

SECTION A – APPLICATION BASED QUESTIONS

Q1. Debugging Pair Sum Code (Failure Due to Incorrect Inner Loop Limit in C)

The following brute force Pair Sum implementation in C stops the inner loop one step too early, which may miss a valid pair involving the final array element. Carefully analyze the nested loop bounds and explain why the complete set of unordered pairs must be explored exactly once. Debug the algorithm so that every valid distinct pair is checked without redundant mirror comparisons. Optimize the corrected implementation for readability and reduced unnecessary work. Analyze the time complexity of the corrected solution and rewrite the final C code with proper comments.

```
int pairSum(int arr[], int n, int target, int *a, int *b) {
    int i, j;
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n - 1; j++) { // ERROR
```

```
        if (arr[i] + arr[j] == target) {
            *a = i;
            *b = j;
            return 1;
        }
    }
    return 0;
}
```

Code of Conduct

I, VIGNESH.Y(192524100), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: VIGNESH.Y

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding; misinterpretation of problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)
Q1 (100 Marks)					

Overall Total (100): _____

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Debugging Pair Sum Code (C)

Aim

To identify and fix the bug in the given brute-force Pair Sum program, optimize it to avoid redundant mirror comparisons, analyze its time complexity, and provide the corrected C program.

Problem Analysis

The given Pair Sum program contains the following incorrect loop bound:

```
for (j = i + 1; j < n - 1; j++) // ERROR
```

The inner loop is meant to scan every index paired with i , from $i+1$ up to the last valid index ($n-1$). Using the bound $j < n - 1$ causes the loop to stop at index $n-2$, so index $n-1$ (the last element of the array) is never used as the value of j . Any valid pair whose second element is the last array element — for example $(i, n-1)$ — is silently skipped, so the function can return 0 (no pair found) even when a valid pair exists.

Why the Inner Loop Must Reach the Last Index

- For a fixed i , j must range over every index strictly greater than i , up to and including $n - 1$, so the correct condition is $j < n$.
- Starting j at $i + 1$ (rather than 0) is what guarantees each unordered pair $\{i, j\}$ is considered exactly once, since pairs with $j < i$ were already generated in an earlier outer-loop iteration as (j, i) .
- Because j never revisits indices $\leq i$, the algorithm never re-checks a pair in mirrored order (e.g., it checks $(0,1)$ but never $(1,0)$ again), which avoids redundant mirror comparisons.
- Together, i from 0 to $n-2$ and j from $i+1$ to $n-1$ enumerate the full set of $C(n,2)$ unordered pairs exactly once, with none missing and none duplicated.

Correct Loop Bound

```
for (j = i + 1; j < n; j++)
```

This ensures j is allowed to take the value $n - 1$, so a pair involving the last array element is always checked when it is a valid candidate.

Systematic Debugging Steps

Step 1 — Locate the incorrect bound.

```
for (j = i + 1; j < n - 1; j++) // ERROR
```

Step 2 — Replace it with the correct bound.

```
for (j = i + 1; j < n; j++)
```

Now every index from $i + 1$ to $n - 1$ is reachable by j , so no valid pair is missed.

Step 3 — Optimize for readability and reduced unnecessary work.

- Return immediately as soon as a matching pair is found (already done), avoiding any further scanning once the target is met.
- Bound the outer loop at $i < n - 1$, since the last element can never be the first element of a still-unchecked pair (it is only ever visited as j).
- Use meaningful variable names and add comments so the intent of each loop bound is explicit, reducing the chance of an off-by-one error being reintroduced.

- No sorting or extra data structures are introduced, since the task asks only to correct and clarify the existing brute-force approach, not to change its asymptotic complexity.

Corrected and Optimized C Program

```
#include <stdio.h>

// Brute-force Pair Sum: finds one pair of indices (a, b) whose
// array values add up to 'target'. Returns 1 and sets *a, *b
// if such a pair exists; returns 0 otherwise.
int pairSum(int arr[], int n, int target, int *a, int *b) {
    int i, j;

    // Outer loop: i is the first element of the pair.
    // i only needs to go up to n - 2, because the last
    // element (index n - 1) is always checked as the
    // second element (j) of some earlier pair, never as i.
    for (i = 0; i < n - 1; i++) {

        // Inner loop: j is the second element of the pair.
        // j starts at i + 1 so each unordered pair {i, j}
        // is examined exactly once (no mirrored repeats),
        // and j must run through n - 1 (FIXED) so the
        // final array element is never skipped.
        for (j = i + 1; j < n; j++) {
            if (arr[i] + arr[j] == target) {
                *a = i;
                *b = j;
                return 1; // stop as soon as a pair is found
            }
        }
    }

    return 0; // no valid pair exists
}

int main(void) {
    int arr[] = {8, 3, 12, 5, 7};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 19;
    int a, b;

    if (pairSum(arr, n, target, &a, &b)) {
        printf("Pair found: arr[%d] + arr[%d] = %d + %d = %d\n",
            a, b, arr[a], arr[b], target);
    } else {
        printf("No pair found for target %d\n", target);
    }

    return 0;
}
```

Sample Output

```
Pair found: arr[2] + arr[3] = 12 + 7 = 19
```

With the original buggy bound ($j < n - 1$), this same input would incorrectly report "No pair found", because index 3 (value 7) is never reached by j .

Time Complexity Analysis

Case	Time Complexity	Explanation
Best Case	$O(1)$	The very first pair checked, $(arr[0], arr[1])$, already sums to the target, so the function returns immediately.
Average Case	$O(n^2)$	On average, roughly half of the $C(n,2)$ pairs must be examined before a match (or no match) is confirmed.
Worst Case	$O(n^2)$	No valid pair exists, or the only valid pair involves the last two elements, so all $C(n,2) = n(n-1)/2$ pairs are examined.

Space Complexity: $O(1)$ — only a fixed number of scalar variables are used; no extra arrays or recursion.

Optimization Analysis

- The fix ($j < n$) does not change the algorithm's asymptotic complexity — it remains $O(n^2)$ — because correctness, not speed, was the bug.
- Bounding the outer loop at $i < n - 1$ removes one redundant outer iteration, since $i = n - 1$ could never begin a new, still-unchecked pair.
- Early return on the first match avoids scanning remaining pairs once the answer is known, which helps the best/average case in practice without changing the worst-case bound.
- Starting j at $i + 1$ (not 0) is itself an optimization over a naive four-loop or 0-indexed inner loop, since it halves the work by never re-examining a pair in reverse order.

Conclusion

The original program failed because the inner loop condition $j < n - 1$ excluded the last valid index, causing the algorithm to miss any pair that included the final array element. Replacing the bound with $j < n$ fixes the bug so every index from $i + 1$ to $n - 1$ is reachable. Keeping j strictly greater than i ensures the complete set of unordered pairs is explored exactly once, with no mirrored repeats. The corrected Pair Sum function remains a brute-force approach with $O(n^2)$ time complexity and $O(1)$ space complexity, and it now returns the correct result for every valid input, including pairs that involve the last element of the array.